

bomblab 报告

姓名：刘镇赫

学号：2024202850

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
14	1	3	2	1	0	1	6

scoreboard 截图：

2024202850	1	3	2	1	0	1	6
------------	---	---	---	---	---	---	---

解题报告

真给我整红温了前几个phase有时忘了break炸掉, 最气人的是secret_phase我尝试./bomb < solution.txt一直炸, 百思不得其解最后手动输发现没问题... 我真无语了(^_^)

phase_1

The autumn leaves, the summer breeze, your shiny hair like mahogany.

```
lea    0x1d40(%rip),%rsi
```

把内存地址0x3180加载到了%rsi

```
call   1cc1 <string_not_equal>
```

调用函数对比输入的字符串和标准答案

因此在call这里设置断点，运行程序后查看%rsi里存的答案地址。

```
b  *(phase_1+0xb)
c
x/s $rsi
```

phase_2

563549 581322 1180408 1237644

调用<__isoc99_sscanf@plt>进行读取输入。
参数指针依次为 0x4(%rsp), 0xc(%rsp), 0x8(%rsp)...(在栈上相邻)

cmp \$0x4, %eax 检查sscanf返回值，必须是4个整数。

矩阵matA.2 在 0x4c9b, 大小为 2*3

矩阵matB.1 在 0x4c4e, 大小为 3*2

0x10(%rsp) 用来临时存放计算结果

循环逻辑：

外层循环(%r11d, 从0到1)：代表结果矩阵的行

每次循环，矩阵A的指针%rdi增加 0xc，指向下一行。

中层循环(%r8d, 从0到1)：代表结果矩阵的列

每次循环，矩阵B的指针%rsi增加 0x4，指向下一列。

内层循环(%rax, 从0到2)：执行点积运算

计算sum += A[row][k] * B[k][col]

计算出的结果是一个 2*2 的矩阵，即4个整数，被顺序写入栈上的 0x10(%rsp)位置

后面的代码负责输入与标准答案的比对。

思路是在比对开始的地方设置断点，运行后输入任意四个整数，当程序停在断点时，寄存器%rbp指向的是正确答案的内存区域。

```
b *0x1518
```

```
run
```

随便输4个整数

```
x/4d $rbp (以十进制查看%rbp指向的4个整数)
```

phase_3

```
5 -283
```

```
xor    %eax, %eax # 将%eax清零
```

```
lea     0x204f(%rip),%rsi # 加载格式字符串 "%d %d"
```

```
call    1150 <__isoc99_sscanf@plt> # 读取输入
```

程序要求输入2个整数，一个存在%rsp(x)，一个存在%rsp+4(y)

确定y范围：

```
cmpl    $0x0,0x4(%rsp) # 检查第二个输入y
```

```
js      157e <phase_3+0x3a> # 如果y < 0，跳到157e(安全)
```

```
call    1f26 <explode_bomb> # 如果y >= 0，炸弹爆炸
```

结论：y < 0

确定x范围：

```
cmpl    $0x7, (%rsp) # 检查第一个输入x
```

```
ja      1622 <phase_3+0xde> # 如果x > 7，跳到爆炸
```

因为ja是无符号比较所以x也必须非负 (不然当成很大的数也炸了)

```
cmpl    $0x5, (%rsp) # 再检查x
```

```
jg      15d6 <phase_3+0x92> # 如果x > 5，跳到爆炸
```

结论：0 <= x <= 5

根据观察发现地址 0x15f1~0x1622 对应的是swicth语句：

前三个跳转指令对应 x = 0, 1, 2,最后都会走到 0x15b0 的call 1f26 <explode_bomb>, 排除;

x = 3 时%eax中计算结果是0，不满足y < 0;

x = 4 时%eax中计算结果是0，不满足y < 0;

x = 5 时%eax中计算结果是0x11b = 283，满足y < 0。

x = 0 时 %eax 中计算结果不是 0x11b = -283, 而是 y = 0;

所以正确答案是 5 -283

phase_4

31 CB

```
lea    0x1ae5(%rip), %rsi # 加载格式字符串 "%d %s"
call   1150 <__isoc99_sscanf@plt> # 读取输入
cmp    $0x2, %eax # 检查是否为2个参数, 否则爆炸
整数存储在 0xc(%rsp), 字符串存储在 0x10(%rsp)
```

计算整数:

```
mov     &0x5, %edi # 设置参数 n = 5
call    1633 <func4_1> # 调用递归函数 func4_1
cmp     %eax, 0xc(%rsp) # 比较返回值与输入的整数
jne     1796 # 不相等就爆炸
```

对func4_1进行分析发现 $f(n) = 2 * f(n-1) + 1$
所以 $f(5) = 31$, 第一个应该输入的整数是31.

计算字符串:

```
lea     0x10(%rsp), %rdi # 加载输入的字符串的地址
call    string_length # 计算长度
cmp     $0x2, %eax # 长度不是2就爆炸
jne     179d
```

```
lea     0x14(%rsp), %rbx # 用于存放生成的字符串
...
mov     $0x42, %r8d # 参数4(r8) = 'B'
mov     $0x43, %ecx # 参数3(rcx) = 'C'
mov     $0x41, %edx # 参数2(rdx) = 'A'
mov     $0xf, %esi # 参数1(rsi) = 15
mov     $0x5, %edi # 参数0(rdi) = 5
call    1659 <func4_2> # 调用生成函数
```

这个函数根据edi和esi决定如何排列传入的字符'A', 'B', 'C', 直到edi=1时将字符写入0x14(%rsp).

递归模拟:

- n = 5, val = 15, chars = A C B
func4_1(4) == 15, 跳转到16b9分支, CB换位;
- n = 4, val = 15, chars = A B C
func4_1(3) = 7 < 15, 且7+1 != 15, 跳转到16d4分支, AC换位;
- n = 3, val = 7, chars = C B A
func4_1(2) = 3 < 7, 且3+1 != 7, 跳转到16d4分支, AC换位;

```

4. n = 2, val = 3, chars = A B C
   func4_1(1) = 1 < 3, 且 1+1 != 3, 跳转到16d4分支, AC换位;
5. n = 1, val = 1, chars = C B A
   n = 1触发条件169f:
   mov     %dl,0x0(%rbp) # 将'C'写入第一个字节
   mov     %cl,0x1(%rbp) # 将'B'写入第二个字节
因此字符串应该是 'CB'

```

所以正确答案 31 CB

phase_5

```
jprqvx
```

```

call    1ca4 <string_length>
cmp     $0x6,%eax
jne     182a <phase_5+0x7a>
输入的字符串长度必须是6

```

题目逻辑:

```

movsbl (%rbx,%rdx,1),%eax # 取出输入字符: char = input[i]
add     $0xf,%eax # char += 15
and     $0xf,%eax # 取低4位作为索引 index = char & 0xf
movzbl (%rcx,%rax,1),%eax # 查找表table[index]
mov     %al,0x1(%rsp,%rdx,1) # 将结果存入栈中

```

获取查找表:

```

17d7: 48 8d 0d 72 1a 00 00 lea     0x1a72(%rip),%rcx      # 3250 <array.0>
17de: 0f be 04 13          movsbl (%rbx,%rdx,1),%eax

```

b *0x17de 设置断点, 随便输入6位字符串, 运行并停止后 x/16cb 0x3250 通过地址查看以下内容:
 <array.0>: 109 'm' 97 'a' 100 'd' 117 'u' 105 'i' 101 'e' 114 'r' 115 's'
 <array.0+8>: 110 'n' 102 'f' 111 'o' 116 't' 118 'v' 98 'b' 121 'y' 108 'l'
 用 x/s 0x3204 查看字符串"flames"

根据规则转换字符后得到结果"jprqvx"

phase_6

```
5 1 3 6 4 2
```

```
call    1fe6 <read_six_numbers> # 读取6个整数, 存入栈 0x10(%rsp) 到 0x24(%rsp)
```

确定输入范围:

地址 1944-1950 的汇编代码如下:

```
mov     %r14,%rbp # %rbp保存当前正在检查的数字的地址(ptr[i])
```

```

mov    (%r14),%eax # 把这个数字读入%eax(val[i])
sub    $0x1,%eax # %eax = val[i] - 1
cmp    $0x5,%eax # 无符号比较val[i] - 1 和 5
ja     187e <phase_6+0x41> # 如果大于5就爆炸

```

输入的每个数字要在[1, 6]之间。

确定范围后查重：

内层循环准备：

```

cmp    $0x5, %r15d # 检查外层循环计数器 i 是否 > 5
jg     18a6 # 如果 i > 5, 说明所有数字都检查完了, 跳出大循环
mov    %r15, %rbx # 把 i 的值给 %rbx, 作为内层循环计数器 j
jmp    1895 # 跳入内层循环

```

内层循环与去重：

```

mov    0x0(%r13,%rbx,4), %eax # 取出 input[j] 的值(%r13 是数组首地址, %rbx 是 j)
cmp    %eax, 0x0(%rbp) # 比较 input[j] 和 input[i] (%rbp指向input[i])
jne    1888 # 如果不相等继续循环
call   explode_bomb # 如果相等, 说明有重复数字, 爆炸

```

内层循环递增：

```

add    $0x1, %rbx # j++
cmp    $0x5, %ebx # 检查 j 是否超过 5 (数组最大下标)
jg     193c # 如果 j > 5, 说明内层比完了, 回到外层循环, 如果没比完, 跳回去继续比下一个

```

外层循环递增(内层循环全部跑完后来到这里)：

```

add    $0x1, %r15 # i++ (准备检查下一个基准数)
add    $0x4, %r14 # 指针后移 4 字节, 指向 input[i+1]
mov    %r14, %rbp # 更新 %rbp 指针
...    # 继续下一轮范围检查和去重

```

根据这个双层循环得知，输入的数字序列必须是1到6的某种排列。

```

mov    0x8(%rsp),%rdx # 循环结束边界
add    $0x18,%rdx
mov    $0x7,%ecx # %ecx = 7
mov    %ecx,%eax # %eax = 7
sub    (%r12),%eax # %eax = 7 - input[i]
mov    %eax,(%r12) # 将结果写回栈中

```

每个输入的数字 x 会变成 7 - x

```

mov    $0x0, %esi # i = 0 (索引)
mov    0x10(%rsp,%rsi,4), %ecx # 取出 input[i] -> 放入 %ecx

lea    0x4934(%rip), %rdx # 加载链表头节点的地址
mov    $0x1, %eax # counter = 1

```

开始寻找节点：

```

cmp    $0x1, %ecx # 检查目标是否是第1个

```

```
jle 18ec # 如果是, 跳出循环
```

没找到, 往下走:

```
mov 0x8(%rdx), %rdx # %rdx = *(%rdx + 8), 即p = p->next
add $0x1, %eax # counter++
cmp %ecx, %eax # 检查是否走完
jne 18e1
```

索引i的循环递增:

```
add $0x1, %rsi
cmp $0x6, %rsi
jne 18cc <phase_6+0x8f>
```

大小比较:

```
mov 0x8(%rbx), %rax # 获取当前节点的 next 节点
mov (%rax), %eax # 获取 next 节点的 value
cmp %eax, (%rbx) # 比较 current.Value vs next.Value
jge 1968 # 如果 Current >= Next, 跳转 (继续检查下一个)
call explode_bomb # 否则爆炸
```

重排链表后每个节点的值必须大于等于后一个节点的值, 降序排列

gdb操作:

在进入phase6后,

1 2 3 4 5 6 # 随便输入

b *phase_6+0xa4 # 停在 lea &node1 后 (18dc), rdx 已设为 node1 地址
c

```
p/x $rdx # ← 确认 node1 addr
set $node = $rdx # 初始 $node = node1
p/d *(int*)$node # node1 值
set $node = *(long*)($node + 8) # next addr
p/d *(int*)$node # node2 值
set $node = *(long*)($node + 8)
p/d *(int*)$node # node3 值
set $node = *(long*)($node + 8)
p/d *(int*)$node # node4 值
set $node = *(long*)($node + 8)
p/d *(int*)$node # node5 值
set $node = *(long*)($node + 8)
p/d *(int*)$node # node6 值
```

把每个node[i]根据value降序排列, 得到重新排列的i列表, 然后再得到 7-i 列表
得到最终答案为 5 1 3 6 4 2

secret_phase

```
5 1 3 6 4 2 hidden
33311
```

如何进入：

```
lea    0x47df(%rip), %rdi # 加载 input_strings 数组基址
...
mov     %eax, %esi # 记录当前的索引
movzbl (%rdi,%rax,1), %ecx # 读取第 i 个字符串的第一个字符
add     $0x1, %rax # 索引 ++
cmp     $0x5, %edx # 循环计数
jg      21b7 <phase_defused+0x56>
...
cmp     $0x6, %edx # 检查是否读完了6个
je      21d6 <phase_defused+0x75>

lea     0x4798(%rip), %rax # input_strings 基址
lea     (%rsi,%rax,1), %rdi # %rsi 是刚才循环算出来的索引（指向最后一行）
read_line 把这一行读进内存，sscanf拿走前六个数字，留下字符串在缓冲区尾部

lea     0x11a5(%rip), %rdi
call    1070 <puts@plt>
...
movslq %esi, %rsi
lea     0x4798(%rip), %rax # 加载输入的字符串数组
lea     (%rsi,%rax,1), %rdi # %rdi = input
lea     0x1416(%rip), %rsi # 加载一个字符串到 %rsi
call    1cc1 <strings_not_equal> # 比较 input vs target
test    %eax, %eax # 检查比较结果
jne     21bc # 如果不相等，跳走（不进隐藏关）
...
2211: call 1bcd <secret_phase> # 如果相等，进入隐藏关
```

目标字符串存储在这里：

```
lea     0x1416(%rip), %rsi # 3601 <array.0+0x3b1>
```

用 x/s 0x3601 查看该地址的字符串得到 "hidden"，放在phase6结果的后面

答案推导：

汇编地址 19c5~1abc：

```
movl    $0xfffffffffe, (%rsp) # stack[0] = -2 (DX[0])
movl    $0xfffffffff, 0x4(%rsp) # stack[1] = -1 (DX[1])
...
movl    $0x1, 0x20(%rsp) # stack[8] = 1 (DY[0])
movl    $0x2, 0x24(%rsp) # stack[9] = 2 (DY[1])
...
```

定义了8种移动方式(国际象棋里马的移动方式)

终点检查：

```
cmp    $0x4, %esi # 检查当前 x 坐标是否为 4
jne    1b34 # 不是则跳走
cmp    $0x7, %edx # 检查当前 y 坐标是否为 7
jne    1b34 # 不是则跳走
目标坐标为(4, 7)

movslq %r9d, %rcx # 获取当前字符串索引
movzbl (%rdi,%rcx,1), %esi # 读取输入字符 char
...
mov    %esi, %r10d
and    $0x7, %r10d # index = char & 7
取低3位作为索引
```

获取棋盘：

b func7

运行停在func7后：

x/128bx &row0 打印从row0开始的128个字节得到期盼。

0表示可以走的路，1表示墙。

经过尝试后得到答案33311可以从(0,0)不触碰障碍走到(4,7)。